

Spring Boot Hour 1 IoC, DI & Spring Boot

Spring Boot — 8-Hour Plan · HOUR 1

What Spring Actually Is (and why Spring Boot exists)

Goal of this hour: after this, when someone says "Spring is an IoC container that does dependency injection," you won't nod blankly — you'll be able to explain it with code.

1.1 Start with the pain, not the framework

Forget Spring for a minute. Imagine you're building a small app with plain Java. You have two classes:

```
class UserRepository {                      // talks to the database
    String getUserById(int id) { return "User" + id; }
}

class UserService {                         // business logic
    private UserRepository repo = new UserRepository(); // ← LOOK HERE

    String getUser(int id) { return repo.getUserById(id); }
}
```

Look at that marked line. `UserService` **creates its own** `UserRepository` using `new`.

Seems innocent. But this one `new` causes three real problems:

Problem 1 — Tight coupling. `UserService` is now glued to this exact `UserRepository` class. Tomorrow your company says "we're switching from MySQL to MongoDB, use `MongoUserRepository` instead." You must **open `UserService` and edit its code**. Now imagine 50 classes all doing `new UserRepository()` — you edit 50 files. One change ripples everywhere. That gluing is called **tight coupling**.

Problem 2 — Hard to test. You want to unit-test `UserService` alone, with a fake repository (so tests don't need a real database). You can't — `UserService` stubbornly builds its own real one inside. There's no door to hand it a fake.

Problem 3 — YOU manage everything. Who creates objects? You. Who decides the order (repository must exist before service)? You. Who wires them together? You. In a big app this becomes a giant, fragile pile of `new` statements.

Analogy: a chef who insists on doing everything himself — growing the vegetables, raising the chickens, forging his own knives. He barely gets time to actually cook. And if the restaurant switches from chicken to paneer, the chef has to rebuild his whole farm.

1.2 The fix — flip who's in control (IoC)

The fix is embarrassingly simple. Instead of `UserService` *creating* its dependency, it **asks for it from outside**:

```
class UserService {
    private UserRepository repo;

    UserService(UserRepository repo) { // "someone, PLEASE HAND me a repository"
        this.repo = repo;
    }

    String getUser(int id) { return repo.getUserById(id); }
}
```

That's it. That's the whole revolution. `UserService` no longer says `new`. It just declares, via its constructor: "I need a `UserRepository` to function. Whoever creates me, give me one."

Now:

- Switching to MongoDB? Hand it a different repository. **`UserService`'s code doesn't change.**
- Testing? Hand it a fake repository. Done.

This flip has a name: **Inversion of Control (IoC)**.

Before: the class controls the creation of what it needs. After: the control is **inverted** — creation happens outside, and the class just receives.

Analogy: the chef stops farming. A kitchen manager now delivers ingredients to his station. The chef just declares his needs — "I need onions and paneer" — and cooks. Switching suppliers? The chef doesn't even know it happened. The chef lost control of supplies — and became MORE productive because of it. That's why "inversion of control" is a good thing despite sounding like a loss.

And the act of *handing in* the dependency (through the constructor) has a name too: **Dependency Injection (DI)**.

IoC is the principle (don't create, receive). **DI is the technique** that implements it (inject through constructor/setter/field).

☛ Interview sentence: "IoC means object creation and wiring is moved out of the class into a container; Dependency Injection is how the container delivers those dependencies — typically via constructor."

1.3 But wait — SOMEONE still has to create the objects...

Right! We removed `new` from `UserService`, but somebody, somewhere, must still do:

```
UserRepository repo = new UserRepository();    // create
UserService service = new UserService(repo);  // inject
```

If YOU write this by hand for 300 classes — figuring out what depends on what, in what order — you've just moved the mess to one giant ugly file.

THIS is the job Spring was born for.

Spring is, at its heart, an IoC container: a smart factory that

1. **scans** your classes,
2. **creates** objects from them (Spring calls these managed objects "**beans**" — just a fancy word for "an object that Spring created and manages"),
3. **figures out the dependency graph** (service needs repository → make repository first),
4. **injects** everything into the right places,
5. and **manages their whole lifecycle** (creation → use → destruction).

You declare. Spring wires. You never write `new` for your own components again.

Analogy continued: Spring is the kitchen manager for your ENTIRE restaurant — it reads every chef's requirement list, procures every ingredient, delivers to every station in the right order, every morning, automatically.

1.4 See it in real Spring code (three tiny annotations)

Here's the same example, Spring-style. Notice how little we write:

```
@Repository                                // "Spring, this class is a bean – a data-access one"
class UserRepository {
    String getUserById(int id) { return "User" + id; }
}

@Service                                    // "Spring, this is a bean too – business logic"
class UserService {
    private final UserRepository repo;

    @Autowired                              // "Spring, INJECT the repository here"
    UserService(UserRepository repo) {       // constructor injection
        this.repo = repo;
    }

    String getUser(int id) { return repo.getUserById(id); }
}
```

What happens at startup, step by step:

1. Spring **scans** your packages and finds classes marked with annotations like `@Repository`, `@Service`, `@Component`, `@Controller` (they all mean "make me a bean" — the different names are just labels for which layer the class belongs to).
2. It sees `UserService`'s constructor needs a `UserRepository`.
3. So it creates the `UserRepository` bean **first**, then creates `UserService`, **injecting** the repository into it.

4. Both beans now live inside the **Spring IoC container** (think of it as Spring's internal warehouse of ready-to-use objects), available wherever needed.

You wrote zero `new`. Zero wiring. You only *declared*.

One note for your interview (it's in your handbook's page 10 too): there are **three injection styles** — constructor, setter, and field injection. **Constructor injection is the recommended one** — dependencies arrive at birth, the field can be `final` (can never be forgotten or changed), and testing is easy.

☛ *"I prefer constructor injection: mandatory dependencies are explicit, fields can be final, and the class can't exist in a half-wired state."*

1.5 So then... what is Spring BOOT? (the question everyone fumbles)

Here's the honest history. Classic Spring solved wiring beautifully — but *setting up* a Spring project was itself painful:

- Long **XML configuration files** describing beans and settings
- Manually choosing ~20 compatible dependency versions
- Installing and configuring an **external Tomcat server**, packaging your app as a WAR file, deploying it into that server
- A day of setup before writing your first real line of code

Spring Boot = Spring with the setup pain removed. Same Spring underneath. Boot adds an opinionated, "just run it" layer:

Pain in classic Spring	Spring Boot's answer
Manual configuration (lots of XML)	Auto-configuration — sees what's on your classpath and configures sensible defaults ("Oh, MySQL driver present? I'll set up a DataSource.")
Choosing 20 compatible libraries	Starters — one line like <code>spring-boot-starter-web</code> pulls in a pre-tested, compatible bundle
External Tomcat + WAR deployment	Embedded server — Tomcat lives <i>inside</i> your app; you run a plain JAR like any Java program
Blind in production	Actuator — free built-in endpoints for health, metrics, monitoring

And the whole thing boots from one tiny class:

```
@SpringBootApplication // ← the magic annotation
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

That one annotation, `@SpringBootApplication`, is secretly **three annotations combined** (memorize this — very common question):

- `@ComponentScan` → "scan this package and below for beans"
- `@Configuration` → "this class can also define beans"
- `@EnableAutoConfiguration` → "set up defaults based on what's on the classpath"

Analogy: Spring is a powerful DSLR camera — amazing photos, but you set ISO, aperture, shutter speed yourself. Spring Boot is the same camera in Auto mode — it reads the scene and picks smart defaults. Same engine; you can still override any setting manually (via `application.properties`) whenever you disagree with a default.

☛ Interview sentence: "Spring Boot is not a replacement for Spring — it's Spring plus auto-configuration, starter dependencies, an embedded server, and Actuator, so you get a production-ready app with near-zero setup."

The comparison table from page 1 of your handbook, now with meaning behind it: Manual vs Auto configuration (auto-config), Complex vs Simple setup (starters), External vs Embedded server, WAR vs JAR (embedded server is why JAR works), More XML vs annotations.

🔧 HOUR 1 DRILL — answer out loud, then check

1. **What problem does `new UserRepository()` inside `UserService` create?** → Tight coupling (change ripples), untestable (can't inject fakes), manual lifecycle management.
2. **What is IoC in one line?** → Object creation/wiring is inverted: moved from the class to the container; classes receive instead of create.
3. **IoC vs DI?** → IoC is the principle; DI is the technique (constructor/setter/field injection) that delivers it.
4. **What is a bean?** → Simply an object that the Spring container creates and manages.
5. **Which injection type do you prefer and why?** → Constructor: explicit, final fields, no half-wired object, easy tests.
6. **Spring vs Spring Boot?** → Boot = Spring + auto-configuration + starters + embedded server + Actuator.
7. **What's inside `@SpringBootApplication`?** → `@ComponentScan` + `@Configuration` + `@EnableAutoConfiguration`.

8. **Why JAR instead of WAR in Boot?** → Server is embedded inside the app, so it runs standalone like a normal Java program.

Next → Hour 2: the request journey — what actually happens when Postman hits your API: DispatcherServlet → Controller → Service → Repository → DB and back.